



Universidad Autónoma de Baja California
Facultad de Ingeniería Mexicali
Ingeniería en Computación

Reporte de Práctica 05
Banderas y Saltos Condicionales

Organización y Arquitectura de Computadoras
Ing. Félix Arredondo Santamaría

Moya Monreal Erick Anselmo — 1110604

Fecha de entrega
10/ABRIL/2026

Mexicali, Baja California
30 de mayo de 2026

Índice

1. Introducción	2
2. Problemática y Justificación	2
3. Objetivos de la práctica	3
4. Marco Teórico	3
4.1. El Registro de Banderas: EFLAGS	3
4.2. Clasificación Completa de las Banderas de EFLAGS	4
4.3. Instrucciones de Comparación y Prueba: CMP y TEST	8
4.4. Instrucciones de Salto Condicional	9
4.5. Control de Flujo sin Estructuras de Alto Nivel	10
5. Procedimiento	11
6. Resultados con pruebas	12
6.1. Ejercicio 1. Las banderas del procesador en MASM x86	12
6.2. Ejercicio 2. Verificación de instrucciones de salto condicional	18
6.3. Ejercicio 3. Contador oscilante en el rango [5, 10]	29
6.4. Ejercicio 4. Determinación de paridad: par o impar	32
7. Conclusiones	36
Referencias	36
A. Repositorio de código	37

1. Introducción

El control de flujo es uno de los pilares fundamentales de cualquier paradigma de programación. En los lenguajes de alto nivel, estructuras como `if`, `while` o `for` abstraen por completo el mecanismo subyacente que las hace posibles. En el lenguaje ensamblador x86 podemos observar como ese mecanismo está implementado por las *banderas de estado* del procesador, que registran el resultado de cada operación aritmética o lógica, y son las instrucciones de *salto condicional* las que leen dichas banderas para decidir si el flujo de ejecución debe desviarse o continuar en secuencia.

En este trabajo se hace uso de las banderas **EFLAGS** de la arquitectura x86, describiendo el propósito y comportamiento de sus banderas, las **banderas de estado** (CF, PF, AF, ZF, SF y OF), la **bandera de control de dirección** (DF) y las **banderas de sistema** (TF, IF, IOPL, NT, RF, VM, AC, VIF, VIP e ID). Asimismo, se implementan y verifican las instrucciones de salto condicional del conjunto de instrucciones MASM, tanto para operandos sin signo como para operandos con signo. Por último, se desarrollan dos programas que aplican estos mecanismos para resolver un contador oscilante de rango acotado y un clasificador de paridad numérica.

2. Problemática y Justificación

La comprensión del control de flujo a nivel de máquina es inseparable del dominio de la programación en ensamblador. En lenguajes de alto nivel, el compilador se encarga de traducir las condiciones lógicas a comparaciones de banderas y saltos condicionales; sin embargo, cuando se trabaja directamente sobre el hardware, el programador debe gestionar este proceso de forma explícita.

Un error frecuente en la programación a bajo nivel consiste en aplicar instrucciones de salto sin considerar si los operandos involucrados son cantidades con o sin signo. Esta confusión es especialmente peligrosa en bucles de control y en la comparación de punteros o índices de memoria.

El desarrollo de esta práctica se justifica como un paso necesario en la transición desde el estudio estático de las instrucciones aritméticas hacia la construcción de programas con lógica de control real. Comprender el papel de cada bandera del registro **EFLAGS** y dominar la semántica de cada salto condicional sienta las bases para implementar estructuras de control complejas, gestionar excepciones aritméticas y optimizar el rendimiento del código a nivel de instrucción.

3. Objetivos de la práctica

Objetivo General

Comprender y aplicar el registro de banderas del procesador x86 y las instrucciones de salto condicional de MASM, para implementar estructuras de control de flujo a nivel de código máquina.

Objetivos Específicos

- **Identificar y describir** el propósito y el comportamiento de las banderas del registro EFLAGS de la arquitectura x86, clasificadas en: banderas de estado (CF, PF, AF, ZF, SF, OF), bandera de control (DF) y banderas de sistema (TF, IF, IOPL, NT, RF, VM, AC, VIF, VIP e ID), señalando cuáles son directamente observables en el depurador de Visual Studio.
- **Verificar empíricamente** el comportamiento de cada instrucción de salto condicional (JE, JNE, JS, JNS, JP, JNP, JO, JNO, JB, JNB, JBE, JNBE, JL, JNL, JLE, JNLE y sus alias), comprobando en cada caso el escenario en que el salto se efectúa y el escenario en que no.
- **Implementar** un contador oscilante utilizando únicamente instrucciones MOV, ADD, SUB y saltos condicionales, que incremente de 5 a 10 y luego decremente de 10 a 5 de forma indefinida.
- **Desarrollar** un programa que determine si un número almacenado en AX es par o impar, empleando operaciones aritméticas y saltos condicionales para almacenar el resultado en una variable de salida.

4. Marco Teórico

4.1. El Registro de Banderas: EFLAGS

La arquitectura x86 de 32 bits incorpora un registro de propósito especial de 32 bits denominado EFLAGS, cuya función es almacenar el estado del procesador y el resultado cualitativo de las operaciones más recientes [1]. A diferencia de los registros de propósito general, EFLAGS no se lee ni se escribe directamente mediante la instrucción MOV; en cambio, sus bits individuales son actualizados automáticamente por el hardware como efecto secundario de instrucciones aritméticas, lógicas, de comparación y de sistema [2].

Cada bit de EFLAGS con significado definido recibe el nombre de *bandera* (*flag*). Estas banderas se dividen en tres categorías principales [1], [3]:

- **Banderas de estado** (*status flags*): reflejan el resultado cualitativo de la última operación aritmética o lógica. Son CF, PF, AF, ZF, SF y OF. La mayor parte de las instrucciones de salto condicional examinan exclusivamente estas banderas.
- **Bandera de control** (*control flag*): modifica el comportamiento del procesador en las instrucciones de cadena (MOVSB, CMPSB, etc.). Es la bandera DF (*Direction Flag*), y se configura con CLD/STD.
- **Banderas de sistema** (*system flags*): controlan el modo de operación del procesador y funciones de bajo nivel del sistema operativo. Incluyen TF, IF, IOPL, NT, RF, VM, AC, VIF, VIP e ID. Su modificación directa está generalmente restringida al nivel de privilegio del *kernel* y no es accesible desde código de usuario ordinario.

El depurador integrado en Microsoft Visual Studio expone en tiempo real ocho de estas banderas mediante un panel de visualización que permite observar su evolución, instrucción a instrucción; las seis banderas de estado (CF, PF, AF, ZF, SF, OF), la bandera de control (DF) y la bandera de sistema IF. El resto de las banderas de sistema no tienen representación directa en ese panel. La Figura 1 muestra la disposición de los bits significativos dentro del registro de 32 bits, coloreados según su categoría.



Figura 1: Disposición de los bits 0–21 del registro **EFLAGS**. Las seis banderas de estado, la de control (DF) y la de sistema IF son las observables en el depurador de Visual Studio.

4.2. Clasificación Completa de las Banderas de EFLAGS

A continuación se presenta la descripción de todas las banderas del registro **EFLAGS**, organizadas por categoría. La Tabla 1 sintetiza la información de cada una, indicando su bit de posición, nombre, abreviatura y accesibilidad desde el depurador de Visual Studio.

Tabla 1: Todas las banderas del registro EFLAGS de la arquitectura x86.

Bit	Abrev	VS label	Nombre	Categoría
<i>Banderas de estado (status flags)</i>				
0	CF	CY/NC	Carry Flag	Estado
2	PF	PE/PO	Parity Flag	Estado
4	AF	AC/NA	Auxiliary Carry Flag	Estado
6	ZF	ZR/NZ	Zero Flag	Estado
7	SF	PL/NG	Sign Flag	Estado
11	OF	OV/NV	Overflow Flag	Estado
<i>Bandera de control</i>				
10	DF	UP/DN	Direction Flag	Control
<i>Banderas de sistema (system flags)</i>				
8	TF	—	Trap Flag	Sistema
9	IF	EI/DI	Interrupt Enable Flag	Sistema
12– 13	IOPL	—	I/O Privilege Level	Sistema
14	NT	—	Nested Task Flag	Sistema
16	RF	—	Resume Flag	Sistema
17	VM	—	Virtual-8086 Mode Flag	Sistema
18	AC	—	Alignment Check Flag	Sistema
19	VIF	—	Virtual Interrupt Flag	Sistema
20	VIP	—	Virtual Interrupt Pending	Sistema
21	ID	—	CPUID Available Flag	Sistema

Banderas de estado

CF — Carry Flag (bit 0): Registra el acarreo o préstamo generado fuera del bit más significativo en operaciones aritméticas *sin signo* [1]. En una suma, $CF = 1$ indica que el resultado no cabe en el registro destino. En una resta o **CMP**, $CF = 1$ indica que el minuendo es menor que el sustraendo. También es afectada por instrucciones de desplazamiento.

PF — Parity Flag (bit 2): Examina el byte menos significativo del resultado y cuenta cuántos bits están en 1 [4]. Si el conteo es par, $PF = 1$; si es impar, $PF = 0$.

AF — Auxiliary Carry Flag (bit 4): Registra el acarreo producido entre el nibble bajo (bits 3–0) y el nibble alto (bits 7–4) de una operación. Es empleada principalmente

por las instrucciones de ajuste decimal DAA y DAS para aritmética BCD.

ZF — Zero Flag (bit 6): Se activa cuando el resultado de la operación es exactamente cero [1]. Es la bandera más consultada por los saltos condicionales.

SF — Sign Flag (bit 7): Es una copia directa del bit más significativo del resultado [4]. $SF = 1$ indica resultado negativo en complemento a dos.

OF — Overflow Flag (bit 11): Se activa cuando una operación aritmética *con signo* produce un resultado fuera del rango representable [1]. Matemáticamente, OF se activa cuando el acarreo *hacia* el bit de signo difiere del acarreo *fuera* del bit de signo.

Bandera de control

DF — Direction Flag (bit 10): Es la única bandera de control. Configura la dirección de avance de las instrucciones de cadena: $DF = 0$ incrementa los punteros ESI/EDI; $DF = 1$ los decrementa [5]. Se gestiona con CLD y STD.

Banderas de sistema

TF — Trap Flag (bit 8): Cuando está activa ($TF = 1$), el procesador genera una interrupción de tipo 1 tras ejecutar cada instrucción, habilitando el modo de ejecución paso a paso (*single-step*). Así funciona el depurador de Visual Studio [2].

IF — Interrupt Enable Flag (bit 9): Controla si el procesador acepta interrupciones enmascarables de hardware [3]. $IF = 1$ permite interrupciones; $IF = 0$ las inhibe. Es la única bandera de sistema visible en el depurador de Visual Studio. Se configura con STI y CLI.

IOPL — I/O Privilege Level (bits 12–13): Campo de 2 bits que indica el nivel de privilegio mínimo requerido para ejecutar instrucciones de E/S (IN, OUT) y para modificar IF [6]. En modo protegido, el sistema operativo establece este campo; el código de usuario normalmente no puede modificarlo.

NT — Nested Task Flag (bit 14): Indica que la tarea actual fue activada mediante la instrucción CALL de cambio de tarea, y que hay una tarea anidada (encadenada). Al ejecutar IRET, el procesador verifica NT para determinar si debe retornar a una tarea previa a través del enlace de la TSS (*Task State Segment*) [2].

RF — Resume Flag (bit 16): Suprime temporalmente las excepciones de punto de interrupción (*breakpoint*) generadas por el registro de depuración DR6 durante la ejecución de la instrucción inmediatamente posterior a una excepción de depuración. Permite que el depurador reanude la ejecución sin entrar en un bucle infinito de excepciones [1].

VM — Virtual-8086 Mode Flag (bit 17): Cuando está activo, el procesador ejecuta código en modo virtual-8086, emulando el entorno del procesador 8086 dentro del modo protegido de 32 bits. Permite ejecutar programas DOS heredados en sistemas operativos modernos [3].

AC — Alignment Check Flag (bit 18): En conjunto con el bit AM del registro de control CR0, habilita la comprobación de alineación en accesos a memoria; si el código de usuario intenta acceder a un dato no alineado (por ejemplo, un **WORD** en una dirección impar), el procesador genera una excepción de verificación de alineación (*Alignment Check*, #AC) [4].

VIF — Virtual Interrupt Flag (bit 19): Es una imagen virtual de IF utilizada cuando el procesador opera en modo virtual-8086 con extensiones VME (*Virtual Mode Extensions*). Permite a las aplicaciones de modo virtual manipular el estado de interrupción sin provocar una trampa al sistema operativo en cada operación [2].

VIP — Virtual Interrupt Pending (bit 20): Indica que hay una o más interrupciones virtuales pendientes de atención. El software de sistema lo examina para determinar cuándo debe simular la entrega de interrupciones a las aplicaciones en modo virtual [2].

ID — ID Flag (bit 21): Si este bit puede ser establecido y borrado por software, el procesador soporta la instrucción **CPUID**, que permite identificar el fabricante, la familia, el modelo y las características del procesador en tiempo de ejecución [1].

Convención de visualización en el depurador

El depurador de Visual Studio expone únicamente las banderas de uso cotidiano en programación, las seis de estado, la de control y la de sistema **IF**. Las muestra con etiquetas fijas de dos letras cuyo valor binario adyacente cambia entre 0 y 1 conforme se ejecutan las instrucciones: por ejemplo, **ZR 1** indica **ZF = 1** (bandera de cero activa); **ZR 0** indica **ZF = 0**. La Tabla 2 resume las etiquetas empleadas.

Tabla 2: Las ocho banderas visibles en el depurador de Visual Studio con sus etiquetas.

Flag	Nombre	Valor=1 / Valor=0	Instrucciones que la usan
OF	Overflow	OV / NV	JO, JNO
DF	Direction	UP / DN	MOVSB, CMPSB, etc.
IF	Interrupt	EI / DI	STI, CLI
SF	Sign	PL / NG	JS, JNS, JL, JNL, etc.
ZF	Zero	ZR / NZ	JE/JZ, JNE/JNZ, etc.
AF	Aux. Carry	AC / NA	DAA, DAS
PF	Parity	PE / PO	JP/JPE, JNP/JPO
CF	Carry	CY / NC	JB, JNB, JBE, etc.

4.3. Instrucciones de Comparación y Prueba: CMP y TEST

Las instrucciones de salto condicional no operan de forma aislada; requieren que alguna instrucción previa haya actualizado las banderas de estado [1]. Las dos instrucciones diseñadas específicamente para este propósito son:

CMP (Compare): Realiza internamente una resta entre los dos operandos (*destino - fuente*), pero *descarta el resultado*; su único efecto es actualizar las banderas ZF, SF, OF y CF. Si ambos operandos son iguales, la resta produce cero y ZF pasa a 1. Si el destino es menor (sin signo), se genera un acarreo y CF pasa a 1 [5].

TEST (Bit Test): Realiza un AND lógico entre los dos operandos, descartando igualmente el resultado y actualizando solo ZF, SF y PF. Es especialmente útil para verificar si un bit específico está en 1, o para comprobar la paridad del contenido de un registro sin alterarlo [4].

La Figura 2 resume el mecanismo de tres etapas que sostiene todo el control de flujo a bajo nivel; una instrucción aritmética, lógica o de comparación fija las banderas, y la instrucción de salto condicional las lee para decidir si bifurca o continúa en secuencia.

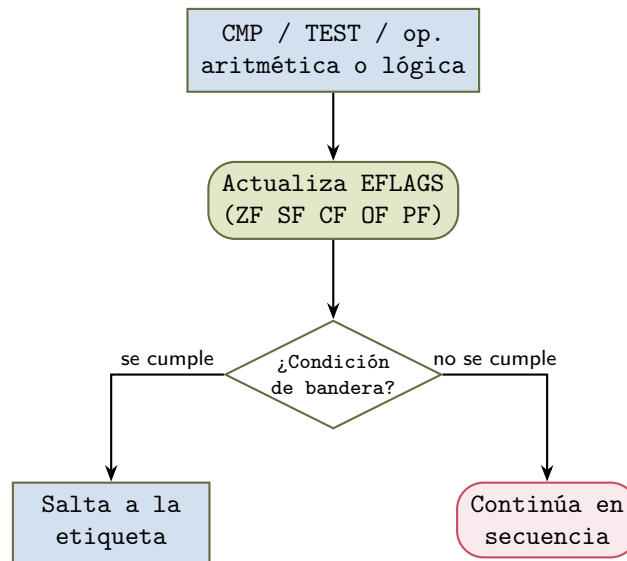


Figura 2: Mecanismo del control de flujo en x86: la instrucción previa fija las banderas de EFLAGS y el salto condicional las evalúa para tomar la bifurcación.

4.4. Instrucciones de Salto Condicional

Las instrucciones de salto condicional evalúan una o varias banderas del registro EFLAGS y, si la condición se cumple, transfieren el control de ejecución a la etiqueta indicada como operando; en caso contrario, la ejecución continúa en la instrucción siguiente [1]. Se dividen en tres categorías funcionales, resumidas en la Tabla 3.

Tabla 3: Resumen de instrucciones de salto condicional en MASM x86.

Instrucción / Alias	Condición	Significado
JE / JZ	ZF=1	Salta si igual / si cero
JNE / JNZ	ZF=0	Salta si no igual / si no cero
JS	SF=1	Salta si resultado negativo
JNS	SF=0	Salta si resultado no negativo
JP / JPE	PF=1	Salta si paridad par
JNP / JPO	PF=0	Salta si paridad impar
JO	OF=1	Salta si hay desbordamiento con signo
JNO	OF=0	Salta si no hay desbordamiento
<i>Salto sin signo (unsigned): examinan CF y ZF</i>		
JB / JNAE	CF=1	Salta si por debajo (<i>below</i>)
JNB / JAE	CF=0	Salta si no por debajo (encima o igual)
JBE / JNA	CF=1 o ZF=1	Salta si por debajo o igual
JNBE / JA	CF=0 y ZF=0	Salta si por encima (<i>above</i>)
<i>Salto con signo (signed): examinan SF, OF y ZF</i>		
JL / JNGE	SF≠OF	Salta si menor que (<i>less</i>)
JNL / JGE	SF=OF	Salta si mayor o igual
JLE / JNG	SF≠OF o ZF=1	Salta si menor o igual
JNLE / JG	SF=OF y ZF=0	Salta si mayor que

4.5. Control de Flujo sin Estructuras de Alto Nivel

En ausencia de instrucciones como `if` o `while`, se construyen las estructuras de control combinando instrucciones de comparación, saltos condicionales y el salto incondicional `JMP` [6]. Un bucle simple equivalente a un `while` se implementa típicamente como: (1) evaluar la condición de parada mediante `CMP` o `TEST`; (2) ejecutar un salto condicional hacia la instrucción posterior al cuerpo del bucle si la condición de terminación se cumple; (3) ejecutar el cuerpo del bucle; y (4) retornar al inicio del bucle con `JMP` [1].

5. Procedimiento

Ejercicio 1. Investigue y describa las banderas disponibles en MASMx86.

Para este ejercicio, se realizó una investigación completa sobre el registro de estado de la arquitectura x86 (EFLAGS). Se identificaron y clasificaron todas las banderas disponibles en sus tres categorías, las **banderas de estado** que reflejan el resultado de las operaciones matemáticas (CF, PF, AF, ZF, SF, OF), la **bandera de control de dirección** (DF) y las **banderas de sistema** que controlan el modo de operación del procesador y funciones del sistema operativo (TF, IF, IOPL, NT, RF, VM, AC, VIF, VIP e ID). Se describió qué significa cada una, cómo cambian según las instrucciones ejecutadas y, en el caso de las banderas de estado, cuáles son observables directamente en el depurador de Visual Studio.

Ejercicio 2. Compruebe el funcionamiento básico (cuando entra y cuando no) de las siguientes instrucciones de salto condicional.

En este punto, escribí varios bloques de código para probar los saltos condicionales que venían en la guía, como JE, JZ, JA, JB, entre otros. Se hizo uso de la instrucción CMP para comparar números y la instrucción TEST para hacer pruebas lógicas. En cada caso, verificamos en el depurador de Visual Studio si el programa de verdad saltaba a la etiqueta indicada cuando se cumplía la condición y si seguía de largo cuando no se cumplía.

Ejercicio 3. Implementar un registro contador que aumente y luego disminuya dentro de un rango específico utilizando únicamente instrucciones de movimiento de registros, operaciones aritméticas y saltos condicionales.

Se desarrolló un programa que hace que un contador oscile entre 5 y 10. Empezamos con el valor 5 en una variable y usamos otra variable para saber si el contador debía subir o bajar. Usamos ADD para sumar y SUB para restar. Después de cada cambio, comparamos el valor con los límites (10 para subir y 5 para bajar) usando saltos condicionales. Cuando el contador llegaba a 10, el programa cambiaba la dirección para empezar a restar, y al llegar a 5, cambiaba otra vez para sumar, creando un ciclo infinito.

Ejercicio 4. Realizar un programa en ensamblador que determine si un número almacenado en un registro es par o impar, haciendo uso exclusivo de operaciones aritméticas y saltos condicionales.

Para saber si un número es par o impar, pusimos un valor en el registro AX y lo dividimos entre 2 usando la instrucción DIV. Como el residuo de la división se guarda automáticamente en el registro DX, usamos una comparación para ver si ese residuo era 0. Si el residuo era 0, el programa usaba un salto condicional para ir a una rutina que guardaba un 1 (par) en el resultado; si no era 0, saltaba a otra rutina para guardar un 0 (impar). Al final, el programa muestra el resultado guardado.

6. Resultados con pruebas

6.1. Ejercicio 1. Las banderas del procesador en MASM x86

Las banderas del registro EFLAGS constituyen el mecanismo mediante el cual la CPU comunica al programador el resultado cualitativo de sus operaciones, así como el estado de funcionamiento del procesador [2]. Se dividen en tres categorías: banderas de estado, bandera de control y banderas de sistema. A continuación se describe cada una en detalle.

Banderas de estado

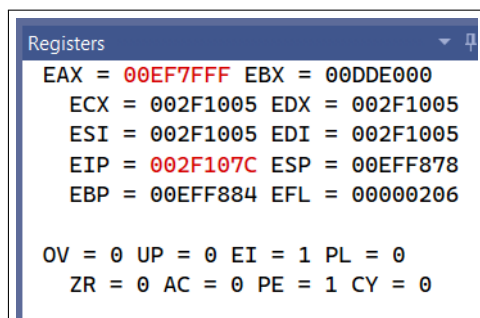
OV / NV — Overflow Flag (OF, bit 11) La bandera de desbordamiento se activa (OV) cuando una operación aritmética con signo produce un resultado que excede el rango representable en el tipo de datos destino [1]. Por ejemplo, en aritmética de 16 bits con signo, el rango válido es $[-32768, +32767]$; sumar 1 a $+32767$ (7FFFh) produce $+32768$ (8000h), que el hardware interpreta como -32768 en complemento a dos, activando OF. Matemáticamente, OF se activa cuando el acarreo al bit de signo y el acarreo fuera del bit de signo son distintos.

Análisis de instrucción

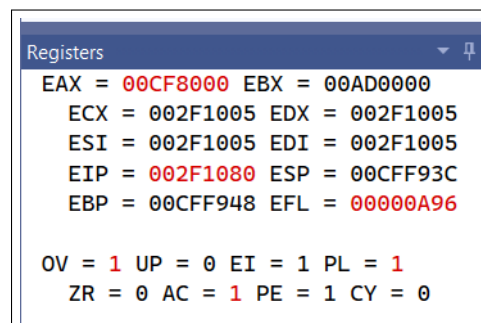
Instrucción que activa OF

```
1  MOV AX, 32767      ; 0x7FFF: maximo positivo en 16 bits
2  ADD AX, 1          ; 32767+1 --> overflow --> OF=1
3  JO  jo_si          ; el salto se produce porque OF=1
```

Descripción: Al sumar 1 al valor máximo positivo representable, el resultado desborda el rango con signo. OF se activa y JO salta.



(a) Antes



(b) Después

Figura 3: Cambio de la bandera OF en el caso de desbordamiento con signo (OV/NV).

PL / NG — Sign Flag (SF, bit 7) La bandera de signo es una copia directa del bit más significativo (*MSB*) del resultado de la última operación [4]. Cuando $SF = 0$ el bit alto del resultado es 0, indicando un valor positivo o cero en interpretación con signo. Cuando $SF = 1$ el resultado es negativo. Es importante destacar que SF no distingue por sí sola entre un resultado correctamente negativo y un desbordamiento; para detectar el desbordamiento debe consultarse OF simultáneamente.

Análisis de instrucción

Instrucción que activa SF

```

1  MOV AX, -5
2  CMP AX, 0      ; -5-0=-5 --> SF=1 (resultado negativo)
3  JS js_si       ; salta porque SF=1 --> NG activo

```

Descripción: `CMP AX, 0` realiza la resta $-5 - 0 = -5$; al ser negativo, el MSB del resultado es 1, activando SF. La instrucción `JS` detecta esta condición y salta.

Registers	
EAX = 002AFFFB	EBX = 004CE000
ECX = 00351005	EDX = 00351005
ESI = 00351005	EDI = 00351005
EIP = 0035103C	ESP = 002AFB60
EBP = 002AFB6C	EFL = 00000246
OV = 0 UP = 0 EI = 1 PL = 0	
ZR = 1 AC = 0 PE = 1 CY = 0	

(a) Antes

Registers	
EAX = 002AFFFB	EBX = 004CE000
ECX = 00351005	EDX = 00351005
ESI = 00351005	EDI = 00351005
EIP = 00351040	ESP = 002AFB60
EBP = 002AFB6C	EFL = 00000282
OV = 0 UP = 0 EI = 1 PL = 1	
ZR = 0 AC = 0 PE = 0 CY = 0	

(b) Después

Figura 4: Cambio de la bandera SF al comparar un valor negativo (PL/NG).

ZR / NZ — Zero Flag (ZF, bit 6) La bandera de cero es, posiblemente, la más utilizada en la programación en ensamblador [1]. Se activa (ZR) cuando el resultado de una operación es exactamente cero. En la práctica, `CMP destino, fuente` activa ZF cuando ambos operandos son iguales (ya que su diferencia es cero), lo que permite implementar comparaciones de igualdad de forma directa mediante `JE/JZ`. También se utiliza como condición de terminación de bucles, cuando el contador llega a cero `JZ` puede redirigir el flujo hacia la salida del ciclo.

Análisis de instrucción

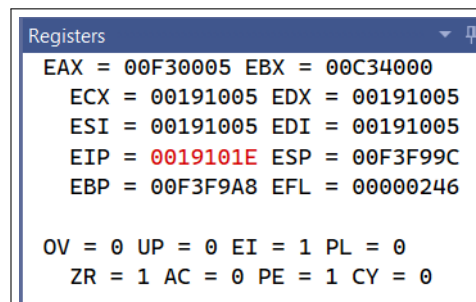
Instrucción que activa ZF

```

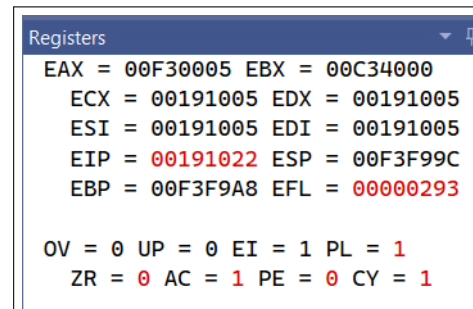
1  MOV AX, 5
2  CMP AX, 10      ; 5 - 10 = -5 --> ZF=0
3  JZ jz_no        ; NO SALTA porque ZF=0 --> NZ activo

```

Descripción: Al comparar valores distintos, la diferencia interna es diferente de cero. ZF no se activa y JE (alias de JZ) no ejecuta el salto.



(a) Antes



(b) Después

Figura 5: Cambio de la bandera ZF al comparar operandos distintos (ZR/NZ).

AC / NA — Auxiliary Carry Flag (AF, bit 4) La bandera de acarreo auxiliar registra el acarreo producido entre los bits 3 y 4 de una operación de 8 bits, es decir, el acarreo del nibble inferior al nibble superior [2]. Su uso primario es en la aritmética BCD (*Binary-Coded Decimal*), donde las instrucciones de ajuste DAA (*Decimal Adjust after Addition*) y DAS (*Decimal Adjust after Subtraction*) la consultan para corregir resultados de sumas y restas sobre dígitos decimales codificados en binario.

PE / PO — Parity Flag (PF, bit 2) La bandera de paridad examina el byte menos significativo del resultado y cuenta cuántos de sus 8 bits están en estado 1 [4]. Si el conteo es par, PF = 1 (depurador muestra PE); si es impar, PF = 0 (depurador muestra PO). Históricamente, esta bandera proviene de la era de las comunicaciones serie, donde la paridad era el método más sencillo de detección de errores de transmisión. En x86, las instrucciones JP/JPE y JNP/JPO permiten bifurcar en función de la paridad.

Análisis de instrucción

Instrucción que activa PF

```

1  MOV AL, 3           ; 00000011b: dos bits en 1 -> paridad par
2  TEST AL, AL         ; AND consigo mismo actualiza PF
3  JP jp_si            ; salta porque PF=1 --> PE activo

```

Descripción: El valor 3 = 00000011b contiene exactamente dos bits en 1 (número par), por lo que PF se activa. JP detecta la paridad par y salta.

Registers	
EAX = 0096FF03	EBX = 0065E000
ECX = 00191005	EDX = 00191005
ESI = 00191005	EDI = 00191005
EIP = 00191062	ESP = 0096FE64
EBP = 0096FE70	EFL = 00000282
OV = 0 UP = 0 EI = 1 PL = 1	
ZR = 0 AC = 0 PE = 0 CY = 0	

(a) Antes

Registers	
EAX = 0096FF03	EBX = 0065E000
ECX = 00191005	EDX = 00191005
ESI = 00191005	EDI = 00191005
EIP = 00191064	ESP = 0096FE64
EBP = 0096FE70	EFL = 00000206
OV = 0 UP = 0 EI = 1 PL = 0	
ZR = 0 AC = 0 PE = 1 CY = 0	

(b) Después

Figura 6: Cambio de la bandera PF en pruebas de paridad (PE/PO).

CY / NC — Carry Flag (CF, bit 0) La bandera de acarreo registra el bit que “se desborda” fuera del límite del registro durante operaciones aritméticas *sin signo* [1]. En una suma, CF = 1 indica que el resultado no cabe en el registro destino (overflow sin signo). En una resta o comparación, CF = 1 indica que el minuendo es menor que el sustraendo (se necesitó un préstamo). También es afectada por instrucciones de desplazamiento de bits, donde recibe el bit que “sale” por el extremo del registro. Las instrucciones JB/JNAE y JNB/JAE la evalúan directamente.

Análisis de instrucción

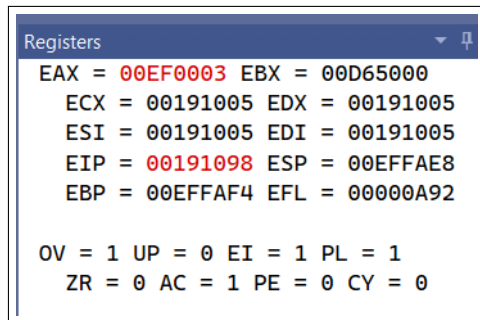
Instrucción que activa CF

```

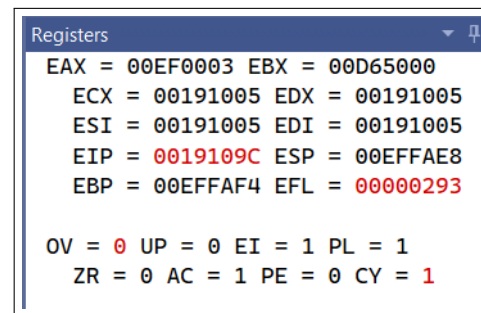
1  MOV AX, 3
2  CMP AX, 5           ; 3-5 requiere préstamo sin signo --> CF=1
3  JB jb_si            ; salta porque CF=1 --> CY activo

```

Descripción: Como 3 es menor que 5 en comparación sin signo, la resta interna genera un acarreo de préstamo. CF se activa y JB (*Jump if Below*) ejecuta el salto.



(a) Antes



(b) Después

Figura 7: Cambio de la bandera CF en una comparación sin signo (CY/NC).

Bandera de control

UP / DN — Direction Flag (DF, bit 10) La bandera de dirección es la única bandera de *control* entre las visibles en el depurador; no refleja el resultado de una operación, sino que configura el comportamiento de las instrucciones de cadena [5]. Cuando $DF = 0$ (depurador muestra UP), instrucciones como `MOVSB` o `CMPSB` incrementan automáticamente `ESI` y `EDI` al procesar cadenas de menor a mayor dirección. Cuando $DF = 1$ (depurador muestra DN), los punteros se decrementan, procesando la cadena de mayor a menor dirección. Se configura con `CLD` (*Clear Direction Flag*, $DF \leftarrow 0$) y `STD` (*Set Direction Flag*, $DF \leftarrow 1$).

Banderas de sistema

Las banderas de sistema controlan el modo de operación del procesador y son gestionadas principalmente por el sistema operativo. Su modificación directa desde código de usuario ordinario está generalmente restringida o prohibida por el nivel de privilegio (*ring*). Estas banderas incluyen:

EI / DI — Interrupt Enable Flag (IF, bit 9) Es la única bandera de sistema que el depurador de Visual Studio muestra directamente [3]. Cuando $IF = 1$ el procesador puede responder a interrupciones enmascarables de hardware (teclado, temporizador, etc.). Cuando $IF = 0$ estas interrupciones son ignoradas. Se gestiona con `STI` y `CLI`.

Trap Flag (TF, bit 8) Cuando $TF = 1$, el procesador genera una excepción de depuración (tipo 1) tras ejecutar cada instrucción, habilitando la ejecución paso a paso (*single-step*) [2].

I/O Privilege Level (IOPL, bits 12–13) Campo de 2 bits que especifica el nivel de privilegio mínimo necesario para ejecutar instrucciones de entrada/salida (IN, OUT) y para modificar la bandera IF [6]. El sistema operativo establece IOPL durante el arranque y el arranque de tareas; el código de usuario en *ring 3* normalmente no puede alterar este campo.

Nested Task Flag (NT, bit 14) Indica que la tarea en ejecución fue invocada mediante un cambio de tarea por *gate* y que existe una tarea encadenada en la estructura TSS (*Task State Segment*) [2]. Al ejecutar IRET con NT = 1, el procesador retorna a la tarea previa a través del enlace en la TSS en lugar de retornar desde una interrupción convencional.

Resume Flag (RF, bit 16) Suprime temporalmente las excepciones de punto de interrupción generadas por los registros de depuración DR0–DR3 al reanudar la ejecución tras una excepción de depuración [1]. Sin esta bandera, el depurador entraría en un bucle infinito al intentar reanudar la ejecución en la misma instrucción que generó la excepción.

Virtual-8086 Mode Flag (VM, bit 17) Cuando está activo, el procesador opera en modo virtual-8086, emulando el entorno del procesador 8086 dentro del modo protegido de 32 bits [3]. Este mecanismo permite ejecutar programas de DOS heredados en sistemas operativos modernos mediante la asignación de una máquina virtual 8086 para cada proceso de ese tipo.

Alignment Check Flag (AC, bit 18) En combinación con el bit AM del registro CR0, habilita la verificación de alineación en accesos a memoria [4]. Si el código de usuario intenta acceder a un dato no alineado en modo protegido (por ejemplo, un DWORD en una dirección no múltiplo de 4), el procesador genera la excepción #AC.

Virtual Interrupt Flag (VIF, bit 19) Imagen virtual de IF empleada cuando el procesador opera en modo virtual-8086 con las extensiones VME (*Virtual Mode Extensions*) activas [2]. Permite a las aplicaciones de modo virtual gestionar interrupciones sin provocar un cambio de contexto al sistema operativo en cada operación sobre IF.

Virtual Interrupt Pending (VIP, bit 20) Indica que hay una o más interrupciones virtuales pendientes de ser entregadas a una aplicación en modo virtual-8086 [2]. El software de sistema examina esta bandera para simular la entrega de interrupciones a las aplicaciones que operan en dicho modo.

ID Flag (ID, bit 21) Si el software puede establecer y borrar este bit, el procesador soporta la instrucción CPUID [1]. Esta instrucción permite identificar al fabricante, la

familia, el modelo y las extensiones de conjunto de instrucciones soportadas (SSE, AVX, etc.) en tiempo de ejecución, y es ampliamente utilizada por compiladores y bibliotecas de software para activar rutas de código optimizadas.

6.2. Ejercicio 2. Verificación de instrucciones de salto condicional

El programa completo de prueba se organiza en bloques consecutivos, cada uno verificando la instrucción en dos escenarios: uno en que el salto *sí* se produce y otro en que *no* se produce. El Código 1 presenta la implementación completa.

```

1      ; --- JE / JZ (Salta si igual / ZF=1) ---
2      MOV AX, 5
3      CMP AX, 5      ; iguales -> ZF=1
4      JE je_si      ; SI SALTA
5  je_si:
6      MOV AX, 5
7      CMP AX, 10     ; distintos -> ZF=0
8      JZ jz_no      ; NO SALTA
9  jz_no:
10     ; --- JNE / JNZ (Salta si no igual / ZF=0) ---
11     MOV AX, 5
12     CMP AX, 10     ; ZF=0
13     JNE jne_si     ; SI SALTA
14  jne_si:
15     MOV AX, 5
16     CMP AX, 5      ; ZF=1
17     JNZ jnz_no     ; NO SALTA
18  jnz_no:
19     ; --- JS (Salta si negativo / SF=1) ---
20     MOV AX, -5
21     CMP AX, 0      ; SF=1 (-5 - 0 = -5)
22     JS js_si      ; SI SALTA
23  js_si:
24     MOV AX, 5
25     CMP AX, 0      ; SF=0
26     JS js_no      ; NO SALTA
27  js_no:
28     ; --- JNS (Salta si no negativo / SF=0) ---
29     MOV AX, 5
30     CMP AX, 0      ; SF=0
31     JNS jns_si     ; SI SALTA
32  jns_si:
33     MOV AX, -5
34     CMP AX, 0      ; SF=1
35     JNS jns_no     ; NO SALTA
36  jns_no:
37     ; --- JP / JPE (Salta si paridad par / PF=1) ---
38     MOV AL, 3      ; 00000011b -> 2 bits en 1 -> par

```

```

39     TEST AL, AL
40     JP    jp_si        ; SI SALTA
41 jp_si:
42     MOV AL, 7          ; 00000111b -> 3 bits en 1 -> impar
43     TEST AL, AL
44     JPE   jpe_no       ; NO SALTA
45 jpe_no:
46     ; --- JNP / JPO (Salta si paridad impar / PF=0) ---
47     MOV AL, 7          ; 00000111b -> impar
48     TEST AL, AL
49     JNP   jnp_si       ; SI SALTA
50 jnp_si:
51     MOV AL, 3          ; 00000011b -> par
52     TEST AL, AL
53     JPO   jpo_no       ; NO SALTA
54 jpo_no:
55     ; --- JO (Salta si overflow / OF=1) ---
56     MOV AX, 32767      ; maximo positivo en 16 bits con signo
57     ADD AX, 1          ; 32767+1 -> overflow -> OF=1
58     JO    jo_si        ; SI SALTA
59 jo_si:
60     MOV AL, 50
61     ADD AL, 10         ; 60, sin overflow -> OF=0
62     JO    jo_no        ; NO SALTA
63 jo_no:
64     ; --- JNO (Salta si no hay overflow / OF=0) ---
65     MOV AL, 50
66     ADD AL, 10         ; OF=0
67     JNO   jno_si       ; SI SALTA
68 jno_si:
69     MOV AL, 127        ; maximo positivo en 8 bits con signo
70     ADD AL, 1          ; 127+1 -> overflow -> OF=1
71     JNO   jno_no       ; NO SALTA
72 jno_no:
73     ; === Saltos SIN SIGNO (unsigned) ===
74     ; --- JB / JNAE (Salta si abajo / CF=1) ---
75     MOV AX, 3
76     CMP AX, 5          ; 3 < 5 -> CF=1
77     JB    jb_si        ; SI SALTA
78 jb_si:
79     MOV AX, 5
80     CMP AX, 3          ; 5 >= 3 -> CF=0
81     JNAE  jnae_no      ; NO SALTA
82 jnae_no:
83     ; --- JNB / JAE (Salta si no abajo / CF=0) ---
84     MOV AX, 5
85     CMP AX, 3          ; CF=0
86     JNB   jnb_si       ; SI SALTA
87 jnb_si:

```

```

88     MOV AX, 3
89     CMP AX, 5          ; CF=1
90     JAE jae_no         ; NO SALTA
91 jae_no:
92     ; --- JBE / JNA (Salta si abajo o igual / CF=1 o ZF=1) ---
93     MOV AX, 5
94     CMP AX, 5          ; ZF=1
95     JBE jbe_si         ; SI SALTA
96 jbe_si:
97     MOV AX, 10
98     CMP AX, 5          ; CF=0, ZF=0
99     JNA jna_no         ; NO SALTA
100 jna_no:
101     ; --- JNBE / JA (Salta si por encima / CF=0 y ZF=0) ---
102     MOV AX, 10
103     CMP AX, 5          ; CF=0, ZF=0
104     JNBE jnbe_si       ; SI SALTA
105 jnbe_si:
106     MOV AX, 5
107     CMP AX, 5          ; ZF=1
108     JA ja_no           ; NO SALTA
109 ja_no:
110     ; === Saltos CON SIGNO (signed) ===
111     ; --- JL / JNGE (Salta si menor / SF!=0F) ---
112     MOV AX, -5
113     CMP AX, 3          ; -5 < 3 -> SF!=0F
114     JL jl_si           ; SI SALTA
115 jl_si:
116     MOV AX, 5
117     CMP AX, 3          ; 5 >= 3 -> SF=0F
118     JNGE jnge_no       ; NO SALTA
119 jnge_no:
120     ; --- JNL / JGE (Salta si no menor / SF=0F) ---
121     MOV AX, 5
122     CMP AX, 3          ; SF=0F
123     JNL jnl_si         ; SI SALTA
124 jnl_si:
125     MOV AX, -5
126     CMP AX, 3          ; SF!=0F
127     JGE jge_no         ; NO SALTA
128 jge_no:
129     ; --- JLE / JNG (Salta si menor o igual / SF!=0F o ZF=1) ---
130     MOV AX, -5
131     CMP AX, 3          ; SF!=0F
132     JLE jle_si         ; SI SALTA
133 jle_si:
134     MOV AX, 10
135     CMP AX, 3          ; ZF=0, SF=0F
136     JNG jng_no         ; NO SALTA

```

```

137 jng_no:
138     ; --- JNLE / JG (Salta si mayor / SF=0F y ZF=0) ---
139     MOV AX, 10
140     CMP AX, 3      ; SF=0F, ZF=0
141     JNLE jnle_si   ; SI SALTA
142 jnle_si:
143     MOV AX, -5
144     CMP AX, 3      ; SF!=0F
145     JG  jg_no      ; NO SALTA
146 jg_no:

```

Listing 1: Código completo del Ejercicio 2: verificación de saltos condicionales.

A continuación se analiza cada grupo de instrucciones de salto, mostrando el estado de las banderas relevantes y el comportamiento observado.

Grupo 1: JE/JZ y JNE/JNZ — Bandera ZF

Análisis de instrucción

Instrucciones

```

1     MOV AX, 5
2     CMP AX, 5      ; ZF=1  -> JE salta (SI)
3     JE  je_si
4
5     MOV AX, 5
6     CMP AX, 10     ; ZF=0  -> JZ no salta (NO)
7     JZ  jz_no

```

Descripción: JE y JZ son alias de la misma instrucción de máquina. Saltan únicamente cuando $ZF = 1$, lo que ocurre si y solo si los dos operandos de CMP son iguales (su diferencia es cero).

Tabla 4: Estado de las banderas para JE/JZ.

Escenario	AX	Operando CMP	ZF (ZR/NZ)	Resultado
Caso salta	5	5	ZR (ZF=1)	Salta a je_si
Caso no salta	5	10	NZ (ZF=0)	Continúa en jz_no

```
Registers
EAX = 008F0005 EBX = 006B6000
ECX = 00191005 EDX = 00191005
ESI = 00191005 EDI = 00191005
EIP = 00191018 ESP = 008FFA70
EBP = 008FFA7C EFL = 00000246

OV = 0 UP = 0 EI = 1 PL = 0
ZR = 1 AC = 0 PE = 1 CY = 0
```

(a) Antes

```
Registers
EAX = 008F0005 EBX = 006B6000
ECX = 00191005 EDX = 00191005
ESI = 00191005 EDI = 00191005
EIP = 00191022 ESP = 008FFA70
EBP = 008FFA7C EFL = 00000293

OV = 0 UP = 0 EI = 1 PL = 1
ZR = 0 AC = 1 PE = 0 CY = 1
```

(b) Después

Figura 8: Verificación de JE/JZ y JNE/JNZ: estado de la bandera ZF (ZR/NZ) antes y después de la comparación.

Grupo 2: JS y JNS — Bandera SF

Análisis de instrucción

Instrucciones

```
1  MOV AX, -5
2  CMP AX, 0      ; SF=1  -> JS salta (SI)
3  JS  js_si
4
5  MOV AX, 5
6  CMP AX, 0      ; SF=0  -> JS no salta (NO)
7  JS  js_no
```

Descripción: JS salta cuando la bandera de signo (SF) está activa, lo que indica que el resultado de CMP fue negativo (el MSB del resultado es 1).

Tabla 5: Estado de las banderas para JS y JNS.

Instrucción	AX	CMP con	SF (PL/NG)	Resultado
JS (salta)	-5	0	NG (SF=1)	Salta
JS (no salta)	5	0	PL (SF=0)	Continúa
JNS (salta)	5	0	PL (SF=0)	Salta
JNS (no salta)	-5	0	NG (SF=1)	Continúa

```

Registers
EAX = 008FFFFB EBX = 006B6000
ECX = 00191005 EDX = 00191005
ESI = 00191005 EDI = 00191005
EIP = 00191040 ESP = 008FFA70
EBP = 008FFA7C EFL = 00000282

OV = 0 UP = 0 EI = 1 PL = 1
ZR = 0 AC = 0 PE = 0 CY = 0
  
```

(a) Antes

```

Registers
EAX = 008F0005 EBX = 006B6000
ECX = 00191005 EDX = 00191005
ESI = 00191005 EDI = 00191005
EIP = 0019104A ESP = 008FFA70
EBP = 008FFA7C EFL = 00000206

OV = 0 UP = 0 EI = 1 PL = 0
ZR = 0 AC = 0 PE = 1 CY = 0
  
```

(b) Después

Figura 9: Verificación de JS/JNS: estado de la bandera SF (PL/NG) antes y después de la comparación.

Grupo 3: JP/JPE y JNP/JPO — Bandera PF

Análisis de instrucción

Instrucciones

```

1      MOV AL, 3          ; 00000011: dos bits en 1-> paridad par
2      TEST AL, AL        ; PF=1
3      JP  jp_si          ; SI SALTA
4
5      MOV AL, 7          ; 00000111: tres bits en 1-> paridad impar
6      TEST AL, AL        ; PF=0
7      JPE jpe_no         ; NO SALTA
  
```

Descripción: TEST AL, AL realiza un AND consigo mismo, actualizando PF según la paridad del byte bajo del resultado (que es el propio AL). 3 = 00000011b tiene dos unos (par), activando PF; 7 = 00000111b tiene tres unos (impar), dejando PF = 0.

Tabla 6: Estado de PF para los valores de prueba.

AL	Binario	Bits en 1	PF (PE/PO)	JP resultado
03h	00000011	2 (par)	PE (PF=1)	Salta
07h	00000111	3 (impar)	P0 (PF=0)	No salta


```

Registers
EAX = 008FFF03 EBX = 006B6000
ECX = 00191005 EDX = 00191005
ESI = 00191005 EDI = 00191005
EIP = 00191064 ESP = 008FFA70
EBP = 008FFA7C EFL = 00000206

OV = 0 UP = 0 EI = 1 PL = 0
ZR = 0 AC = 0 PE = 1 CY = 0

```

(a) Antes

```

Registers
EAX = 008FFF07 EBX = 006B6000
ECX = 00191005 EDX = 00191005
ESI = 00191005 EDI = 00191005
EIP = 0019106A ESP = 008FFA70
EBP = 008FFA7C EFL = 00000202

OV = 0 UP = 0 EI = 1 PL = 0
ZR = 0 AC = 0 PE = 0 CY = 0

```

(b) Después

Figura 10: Verificación de JP/JPE y JNP/JPO: estado de la bandera PF (PE/PO) antes y después de la comparación.

Grupo 4: JO y JNO — Bandera OF

Análisis de instrucción

Instrucciones

```

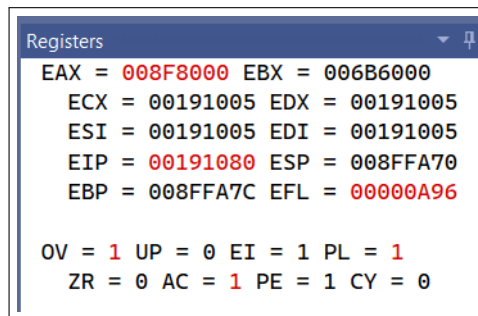
1      MOV AX, 32767      ; 0x7FFF: maximo positivo en 16 bits con
                        ; signo
2      ADD AX, 1          ; 32767+1 = -32768 -> overflow -> OF=1
3      JO  jo_si          ; SI SALTA
4
5      MOV AL, 50
6      ADD AL, 10         ; 60, sin overflow -> OF=0
7      JO  jo_no          ; NO SALTA

```

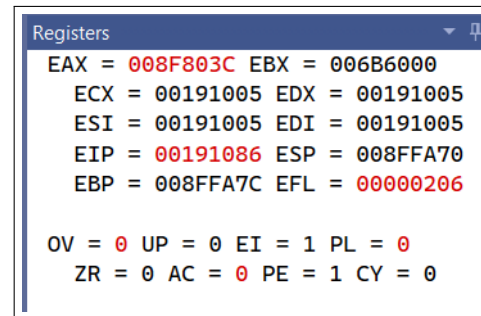
Descripción: El primer caso provoca un desbordamiento con signo al incrementar el valor máximo de un WORD con signo. En el segundo caso, $50 + 10 = 60$ cabe perfectamente en 8 bits con signo, por lo que OF no se activa.

Tabla 7: Estado de OF para los casos de prueba de JO/JNO.

Operación	Resultado	¿Overflow?	OF (OV/NV)	Resultado salto
32767 + 1 (WORD)	-32768 (erróneo)	Sí	OV	JO salta
50 + 10 (BYTE)	60 (correcto)	No	NV	JO no salta
50 + 10 (BYTE)	60 (correcto)	No	NV	JNO salta
127 + 1 (BYTE)	-128 (erróneo)	Sí	OV	JNO no salta



(a) Antes



(b) Después

Figura 11: Verificación de JO/JNO: estado de la bandera OF (OV/NV) antes y después de la operación.

Grupo 5: Saltos sin signo — JB/JNAE, JNB/JAE, JBE/JNA, JNBE/JA

Los saltos sin signo examinan la bandera de acarreo (CF) y, en el caso de JBE/JNBE, también ZF. Son apropiados para comparar índices de arreglo, punteros o cualquier cantidad que conceptualmente no puede ser negativa.

Análisis de instrucción

Instrucciones representativas

```

1  MOV AX, 3
2  CMP AX, 5      ; 3 < 5 -> CF=1
3  JB  jb_si      ; SI SALTA (3 esta por debajo de 5)
4
5  MOV AX, 5
6  CMP AX, 3      ; 5 >= 3 -> CF=0
7  JNAE jnae_no   ; NO SALTA (5 no esta por debajo de 3)

```

Descripción: JB (*Jump if Below*) salta si CF = 1, lo que indica que la resta interna de CMP requirió un préstamo, es decir, que el operando destino es menor en sentido sin signo que el operando fuente.

Tabla 8: Resumen de saltos sin signo y sus condiciones de bandera.

Instrucción	CF	ZF	Condición de salto
JB / JNAE	1	—	Salta si por debajo (sin signo)
JNB / JAE	0	—	Salta si no por debajo (encima o igual)
JBE / JNA	1 o —	— o 1	Salta si por debajo o igual
JNBE / JA	0	0	Salta si por encima

```
Registers
EAX = 008F0003 EBX = 006B6000
ECX = 00191005 EDX = 00191005
ESI = 00191005 EDI = 00191005
EIP = 0019109C ESP = 008FFA70
EBP = 008FFA7C EFL = 00000293

OV = 0 UP = 0 EI = 1 PL = 1
ZR = 0 AC = 1 PE = 0 CY = 1
```

(a) Antes

```
Registers
EAX = 008F0005 EBX = 006B6000
ECX = 00191005 EDX = 00191005
ESI = 00191005 EDI = 00191005
EIP = 001910A6 ESP = 008FFA70
EBP = 008FFA7C EFL = 00000202

OV = 0 UP = 0 EI = 1 PL = 0
ZR = 0 AC = 0 PE = 0 CY = 0
```

(b) Después

Figura 12: Verificación de los saltos sin signo (JB, JNB, JBE, JA): estado de CF y ZF antes y después.

Grupo 6: Saltos con signo — JL/JNGE, JNL/JGE, JLE/JNG, JNLE/JG

Los saltos con signo evalúan la relación entre SF y OF. Cuando ambas banderas tienen el mismo valor ($SF = OF$), la comparación produjo un resultado no negativo desde el punto de vista con signo; cuando difieren ($SF \neq OF$), el resultado es negativo con signo. Esta distinción se usa al comparar valores que pueden ser negativos.

Análisis de instrucción

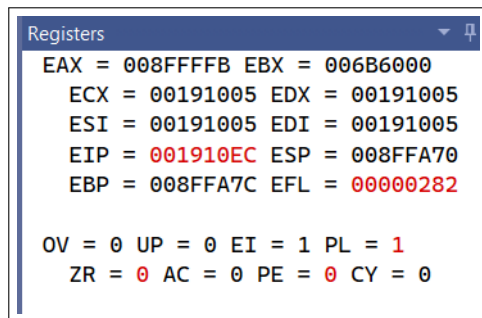
Instrucciones representativas

```
1  MOV AX, -5
2  CMP AX, 3      ; -5 < 3 -> SF!=OF -> JL salta (SI)
3  JL  jl_si
4
5  MOV AX, 5
6  CMP AX, 3      ; 5 >= 3 -> SF=OF -> JNGE no salta (NO)
7  JNGE jnge_no
```

Descripción: La comparación -5 vs 3 produce una resta $-5 - 3 = -8$, cuyo resultado negativo activa SF. Como no hay desbordamiento, $OF = 0$. La desigualdad $SF \neq OF$ confirma que $-5 < 3$ con signo, y JL salta.

Tabla 9: Resumen de saltos con signo y sus condiciones de bandera.

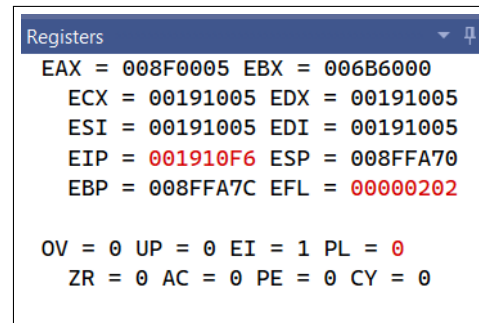
Instrucción	SF vs OF	ZF	Condición de salto
JL / JNGE	$SF \neq OF$	—	Salta si menor que (con signo)
JNL / JGE	$SF = OF$	—	Salta si mayor o igual (con signo)
JLE / JNG	$SF \neq OF$ o	1	Salta si menor o igual (con signo)
JNLE / JG	$SF = OF$ y	0	Salta si mayor que (con signo)



```
Registers
EAX = 008FFFFB EBX = 006B6000
ECX = 00191005 EDX = 00191005
ESI = 00191005 EDI = 00191005
EIP = 001910EC ESP = 008FFA70
EBP = 008FFA7C EFL = 00000282

OV = 0 UP = 0 EI = 1 PL = 1
ZR = 0 AC = 0 PE = 0 CY = 0
```

(a) Antes



```
Registers
EAX = 008F0005 EBX = 006B6000
ECX = 00191005 EDX = 00191005
ESI = 00191005 EDI = 00191005
EIP = 001910F6 ESP = 008FFA70
EBP = 008FFA7C EFL = 00000202

OV = 0 UP = 0 EI = 1 PL = 0
ZR = 0 AC = 0 PE = 0 CY = 0
```

(b) Después

Figura 13: Verificación de los saltos con signo (JL, JNL, JLE, JG): relación entre SF, OF y ZF antes y después.

Resumen de instrucciones de salto condicional

Tabla 10: Tabla de verificación de todas las instrucciones de salto probadas.

Instrucción	Bandera(s)	Operandos de prueba	¿Salta?	Observado
JE	ZF=1	AX=5, CMP 5	Sí	✓
JZ	ZF=1	AX=5, CMP 10	No	✓
JNE	ZF=0	AX=5, CMP 10	Sí	✓
JNZ	ZF=0	AX=5, CMP 5	No	✓
JS	SF=1	AX=-5, CMP 0	Sí	✓
JS	SF=0	AX=5, CMP 0	No	✓
JNS	SF=0	AX=5, CMP 0	Sí	✓
JNS	SF=1	AX=-5, CMP 0	No	✓
JP	PF=1	AL=03h (par)	Sí	✓
JPE	PF=0	AL=07h (impar)	No	✓
JNP	PF=0	AL=07h (impar)	Sí	✓
JPO	PF=1	AL=03h (par)	No	✓
JO	OF=1	AX=7FFF+1	Sí	✓
JO	OF=0	AL=50+10	No	✓
JNO	OF=0	AL=50+10	Sí	✓
JNO	OF=1	AL=7F+1	No	✓
JB	CF=1	AX=3, CMP 5	Sí	✓
JNAE	CF=0	AX=5, CMP 3	No	✓
JNB	CF=0	AX=5, CMP 3	Sí	✓
JA	CF=1	AX=3, CMP 5	No	✓
JBE	ZF=1	AX=5, CMP 5	Sí	✓
JNA	CF=0,ZF=0	AX=10,CMP 5	No	✓
JNBE	CF=0,ZF=0	AX=10,CMP 5	Sí	✓
JA	ZF=1	AX=5, CMP 5	No	✓
JL	SF≠OF	AX=-5,CMP 3	Sí	✓
JNGE	SF=OF	AX=5, CMP 3	No	✓
JNL	SF=OF	AX=5, CMP 3	Sí	✓
JGE	SF≠OF	AX=-5,CMP 3	No	✓
JLE	SF≠OF	AX=-5,CMP 3	Sí	✓
JNG	SF=OF,ZF=0	AX=10,CMP 3	No	✓
JNLE	SF=OF,ZF=0	AX=10,CMP 3	Sí	✓
JG	SF≠OF	AX=-5,CMP 3	No	✓

6.3. Ejercicio 3. Contador oscilante en el rango [5, 10]

El programa implementa un contador bidireccional que aumenta de 5 a 10 y luego disminuye de vuelta a 5, repitiendo el ciclo de manera indefinida. Para ello se utilizan dos variables auxiliares, `contador` (inicializado en 5) y `direccion` (inicializado en 1, que indica incremento), y únicamente las instrucciones permitidas, `MOV`, `ADD`, `SUB`, `CMP` y saltos condicionales.

Conceptualmente, el algoritmo es una máquina de dos estados gobernada por la variable `direccion`, mientras el contador no toca un límite, permanece en el estado actual sumando o restando; al alcanzar el extremo, invierte la dirección. La Figura 14 ilustra esta lógica.

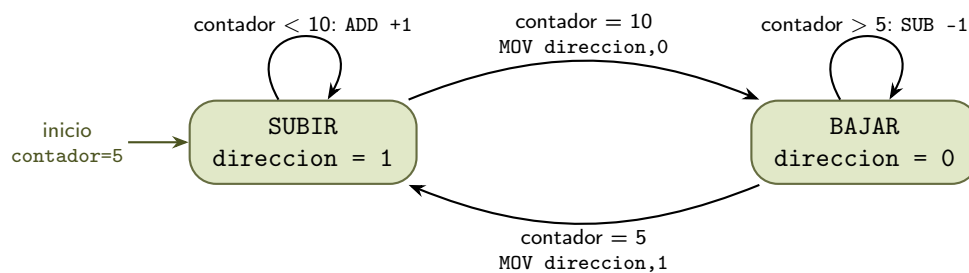


Figura 14: Máquina de dos estados del contador oscilante: cada estado itera con su operación aritmética y conmuta al opuesto cuando `contador` alcanza el límite correspondiente.

```

1  .data
2      contador    WORD 5
3      direccion   WORD 1
4      ultimo      WORD 0
5
6  .code
7  ciclo:
8      CMP     direccion, 1
9      JNE     bajar
10
11  subir:
12      ADD     contador, 1
13      CMP     contador, 10
14      JNE     mostrar
15      MOV     direccion, 0
16      JMP     mostrar
17
18  bajar:
19      SUB     contador, 1
20      CMP     contador, 5
21      JNE     mostrar
22      MOV     direccion, 1
23
24  mostrar:

```

```

25     MOV     AX, contador
26     MOV     ultimo, AX
27     JMP     ciclo

```

Listing 2: Código del Ejercicio 3: contador oscilante (5, 10).

Análisis del flujo de control

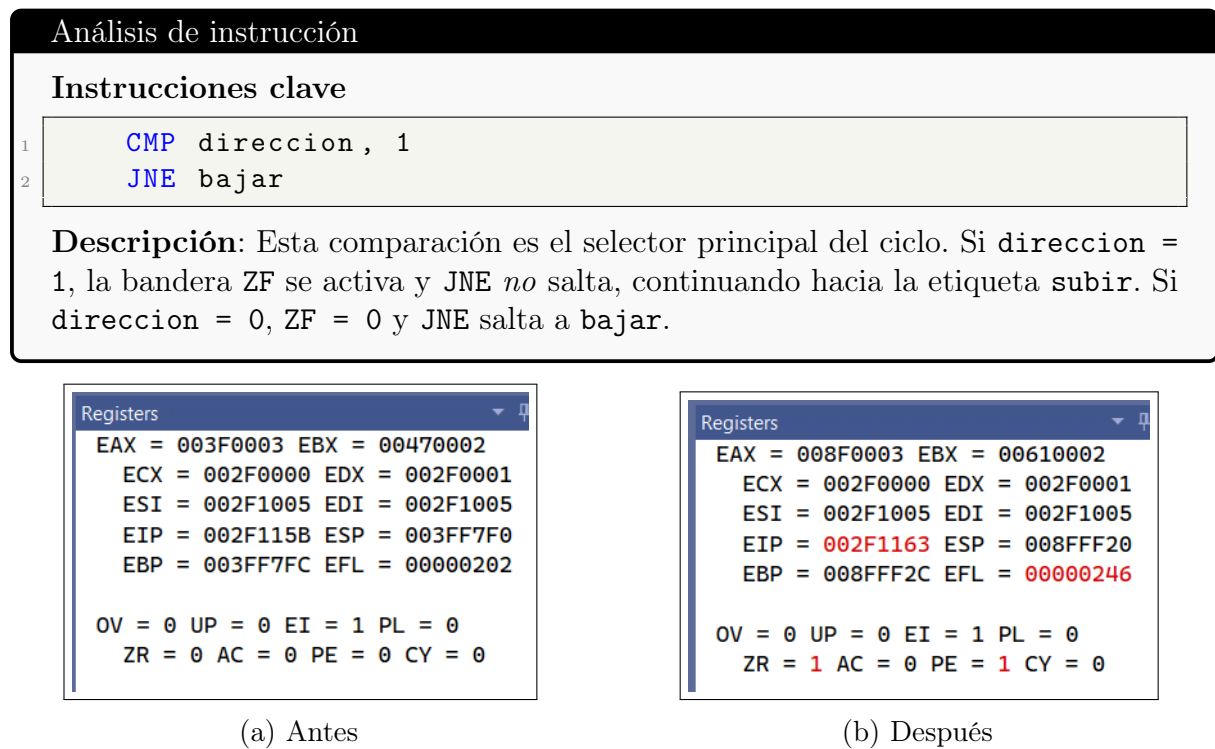
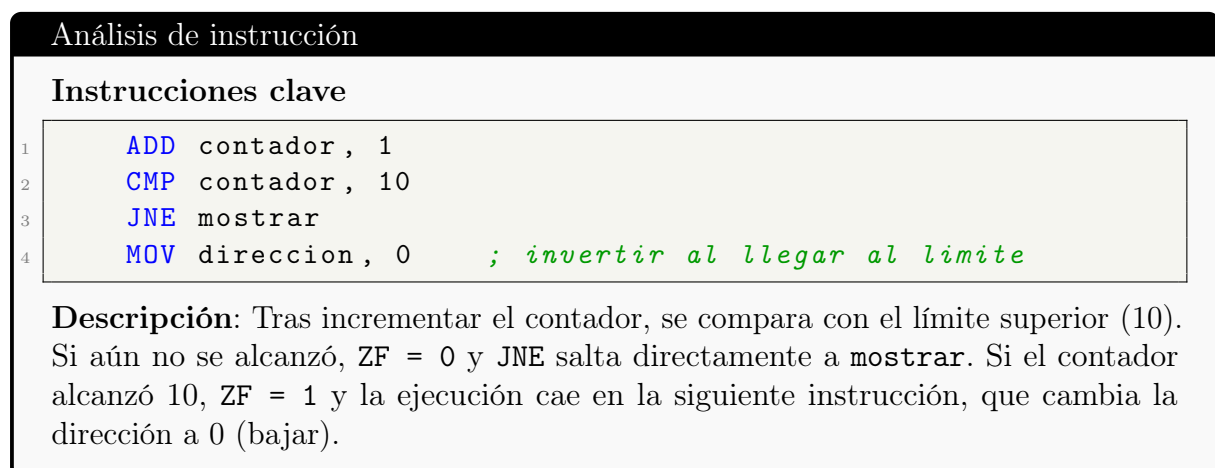
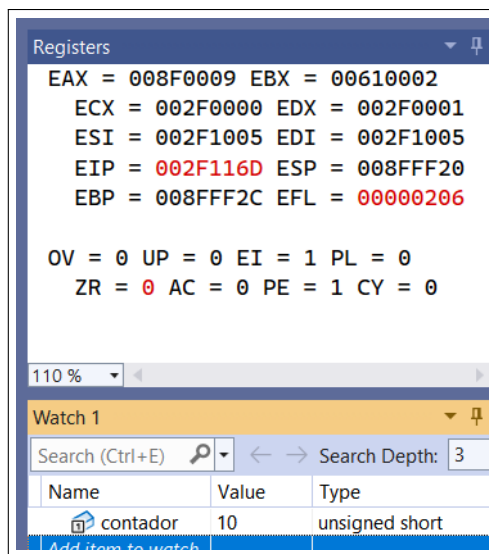
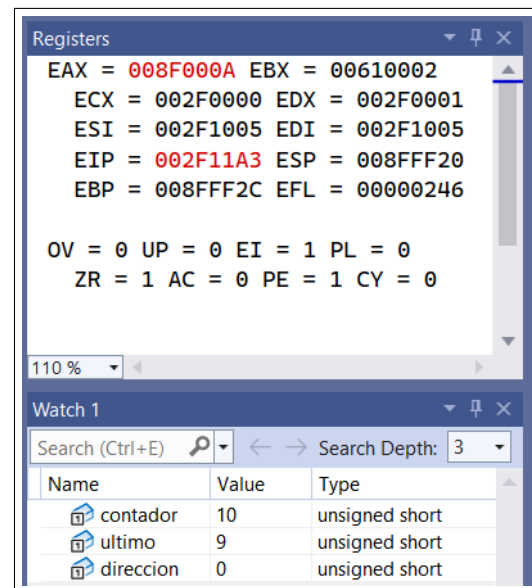


Figura 15: Selector de dirección del contador: `CMP direccion,1` y la decisión de JNE según ZF, antes y después.





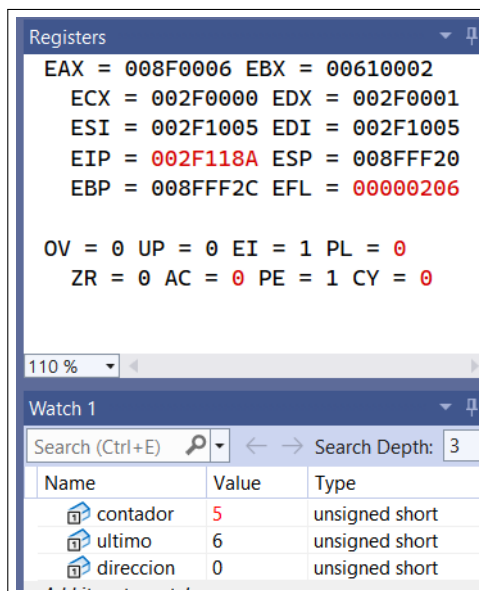
(a) Antes



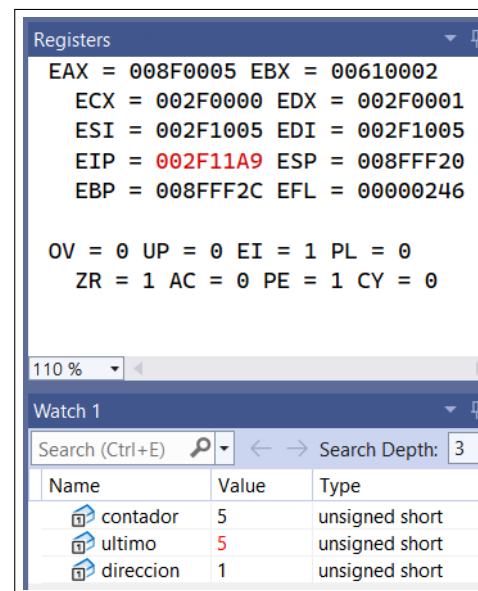
(b) Después

Figura 16: Contador al alcanzar el límite superior (10): cambio de ZF y actualización de direccion a 0, antes y después.

La lógica simétrica se aplica en la rama **bajar**, tras decrementar, si el contador llega a 5 se invierte la dirección a 1 (subir) para reiniciar el ciclo ascendente.



(a) Antes



(b) Después

Figura 17: Contador al alcanzar el límite inferior (5): cambio de ZF y actualización de direccion a 1, antes y después.

Tabla 11: Traza de la secuencia de valores de **contador** en los primeros dos ciclos.

Iteración	contador	direccion	Acción
0	5	1	Inicio
1	6	1	Subir
2	7	1	Subir
3	8	1	Subir
4	9	1	Subir
5	10	0	Llegó a 10, cambia a bajar
6	9	0	Bajar
7	8	0	Bajar
8	7	0	Bajar
9	6	0	Bajar
10	5	1	Llegó a 5, cambia a subir
:	:	:	(ciclo indefinido)

6.4. Ejercicio 4. Determinación de paridad: par o impar

El programa determina si el número almacenado en la variable **numero** (inicializada en 8 para este ejemplo) es par o impar. El mecanismo empleado es la división entera entre 2 con la instrucción **DIV**, el residuo obtenido en **DX** es el discriminante de paridad, ya que cualquier número par produce residuo 0 al dividirse entre 2, y cualquier número impar produce residuo 1.

```

1  .data
2      numero      WORD 8
3      resultado   WORD 0
4
5  .code
6      MOV AX, numero
7      SUB DX, DX
8      MOV BX, 2
9      DIV BX
10     CMP DX, 0
11     JNE impar
12
13 par:
14     MOV CX, 1
15     JMP guardar
16
17 impar:
18     MOV CX, 0
19
20 guardar:
21     MOV resultado, CX

```

Listing 3: Código del Ejercicio 4: clasificación par/impar.

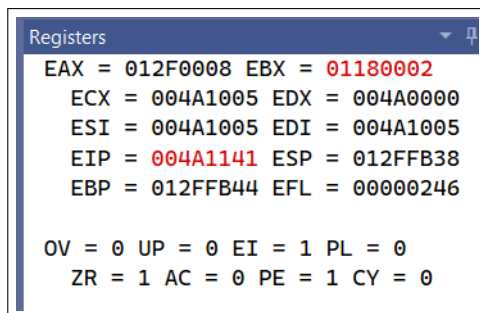
Análisis del algoritmo

Análisis de instrucción

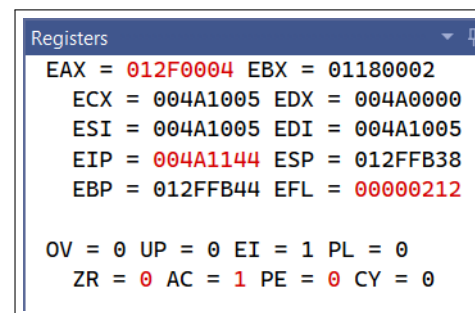
Instrucciones clave

```
1  SUB DX, DX
2  MOV BX, 2
3  DIV BX
```

Descripción: La instrucción `DIV BX` realiza la división sin signo del par `DX:AX` entre `BX`. Es esencial limpiar `DX` antes de la operación; de lo contrario, el dividendo sería un número de 32 bits con basura en la parte alta, lo que provocaría un desbordamiento de cociente (*divide overflow*) si `DX` contiene cualquier valor diferente de cero.



(a) Antes



(b) Después

Figura 18: Registros durante la división: preparación de `DX:AX`, divisor en `BX` y resultado en `AX/DX`, antes y después de `DIV`.

Análisis de instrucción

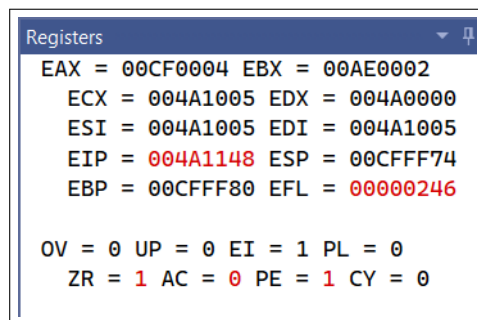
Instrucciones clave

```

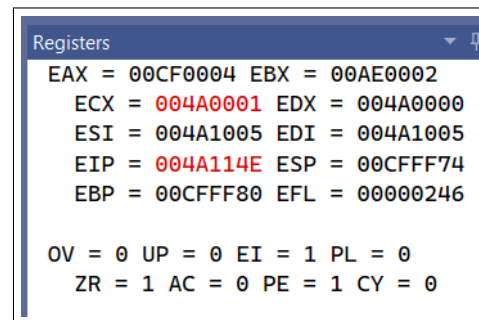
1      CMP DX, 0
2      JNE impar
3  par:
4      MOV CX, 1
5      JMP guardar
6  impar:
7      MOV CX, 0
8  guardar:
9      MOV resultado, CX

```

Descripción: Tras la división, DX contiene el residuo (0 o 1). La comparación con 0 activa ZF si el número es par. JNE salta a *impar* si $ZF = 0$ (residuo $\neq 0$). Si el número es par, la ejecución cae en *par*, carga 1 en CX y salta incondicionalmente a *guardar* con JMP para evitar ejecutar el bloque *impar*.

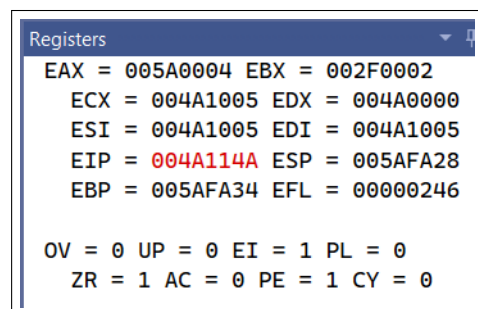


(a) Antes

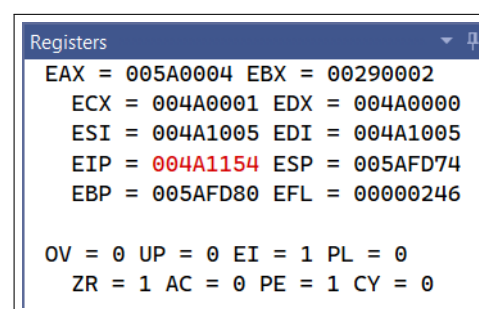


(b) Después

Figura 19: Decisión par/impar: estado de ZF tras `CMP DX,0` y la ruta tomada por `JNE`, antes y después.



(a) Antes



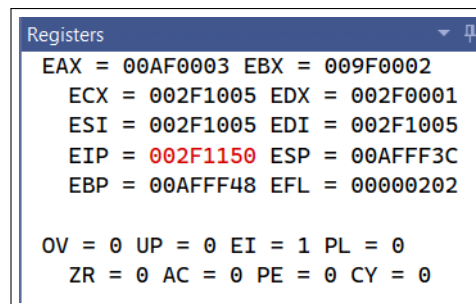
(b) Después

Figura 20: Estado de los registros en el depurador tras ejecutar el bloque *par* para `numero = 8`.

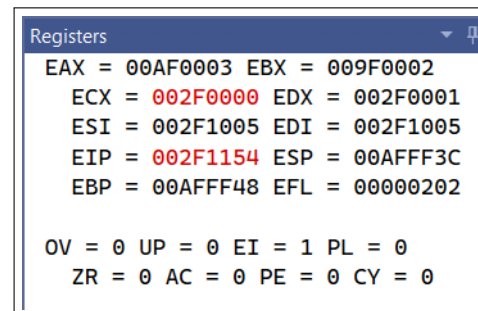
Tabla 12: Estado de los registros tras DIV BX para numero = 8.

AX (entrada)	BX (divisor)	AX (cociente)	DX (residuo)	Clasificación
0008h (8)	0002h (2)	0004h (4)	0000h (0)	Par $\Rightarrow$ resultado = 1

Para validar el comportamiento con un número impar, se cambia la variable **numero** a 7 y se repite la ejecución.



(a) Antes



(b) Después

Figura 21: Estado de los registros en el depurador tras ejecutar el bloque **impar** para **numero** = 7.

Tabla 13: Estado de los registros tras DIV BX para numero = 7.

AX (entrada)	BX (divisor)	AX (cociente)	DX (residuo)	Clasificación
0007h (7)	0002h (2)	0003h (3)	0001h (1)	Impar $\Rightarrow$ resultado = 0

Tabla 14: Resumen de la verificación par/impar para ambos casos de prueba.

numero	DX (residuo)	ZF (ZR/NZ)	Salto tomado	resultado
8 (par)	0	ZR (ZF=1)	Cae en par	1
7 (impar)	1	NZ (ZF=0)	Salta a impar	0

7. Conclusiones

Esta práctica permitió comprender el mecanismo de control de flujo, cerrando el ciclo entre las operaciones aritméticas de la práctica anterior y su aplicación en decisiones condicionales. El uso de las banderas del registro **EFLAGS** constituye un canal de comunicación entre el hardware y el programador, las de estado capturan características cualitativas del resultado de cada instrucción, la de control modifica el comportamiento de las operaciones de cadena, y las de sistema permiten al sistema operativo gestionar el modo de ejecución del procesador, las interrupciones y los mecanismos de depuración.

La verificación sistemática de los dieciséis grupos de saltos condicionales fue suficiente para entenderlas. Aplicar **JL** donde debería usarse **JB**, o viceversa, produciría un comportamiento incorrecto en presencia de números negativos o valores grandes sin signo; esta es una fuente habitual de *bugs* difíciles de diagnosticar.

Los Ejercicios 3 y 4 utilizaron estructuras de control complejas como los bucles con condición de cambio y la bifurcación por resultado aritmético, que son perfectamente implementables con las instrucciones más básicas del conjunto de instrucciones **MASMx86**. La ausencia de estructuras de alto nivel, lejos de ser una limitación, pues exhibe con transparencia el mecanismo subyacente que los compiladores modernos generan automáticamente, una comparación de banderas seguida de un salto condicional.

Referencias

- [1] K. R. Irvine, *Assembly Language for x86 Processors*, 8th. Hoboken, NJ: Pearson, 2020, ISBN: 978-0-13-538165-6.
- [2] W. Stallings, *Computer Organization and Architecture: Designing for Performance*, 11th. Hoboken, NJ: Pearson Education, Inc., 2019, ISBN: 978-0-13-499719-3.
- [3] A. S. Tanenbaum y T. Austin, *Structured Computer Organization*, 6th. Pearson, 2013.
- [4] A. Elahi, *Computer Systems: Digital Design, Fundamentals of Computer Architecture and ARM Assembly Language*, 2nd. Cham, Switzerland: Springer Nature Switzerland AG, 2022, ISBN: 978-3-030-93448-4. DOI: [10.1007/978-3-030-93449-1](https://doi.org/10.1007/978-3-030-93449-1).
- [5] K. R. Irvine, *Assembly Language for x86 Processors*, 6th. Upper Saddle River, NJ: Prentice Hall, 2011.
- [6] D. A. Patterson y J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, ARM. Morgan Kaufmann, 2016.
- [7] J. L. Peterson, *Computer Organization and Assembly Language Programming*, Digital Edition. James L. Peterson, 2018.

A. Repositorio de código

El código **Ensamblador** completo de los ejercicios realizados durante esta práctica se encuentra disponible en el siguiente repositorio de GitHub:

<https://github.com/ApoloEM/OyAdeC/tree/main/practica5>